PyTorch Basics

This notebook covers the fundamental concepts of PyTorch, including tensors, operations, and computational graphs.

In [ ]:
```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Check PyTorch version
print(f"PyTorch version: {torch.__version__}")
```

1. Tensors

Tensors are the fundamental data structure in PyTorch, similar to NumPy arrays but with additional capabilities like GPU acceleration and automatic differentiation.

1.1 Creating Tensors

In [ ]:
```
# Create a tensor from a Python list
x = torch.tensor([1, 2, 3, 4])
print(f"Tensor from list: {x}")

# Create a 2D tensor (matrix)
matrix = torch.tensor([[1, 2], [3, 4]])
print(f"\nMatrix:\n{matrix}")

# Create tensors with specific data types
float_tensor = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float32)
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64)
print(f"\nFloat tensor: {float_tensor}")
print(f"Integer tensor: {int_tensor}")
```

In [ ]:
```
# Create tensors with specific shapes
zeros = torch.zeros(3, 4)
ones = torch.ones(2, 3)
rand = torch.rand(2, 2)  # Uniform distribution [0, 1)
randn = torch.randn(2, 2)  # Normal distribution (mean=0, std=1)

print(f"Zeros tensor:\n{zeros}")
print(f"\nOnes tensor:\n{ones}")
print(f"\nRandom uniform tensor:\n{rand}")
print(f"\nRandom normal tensor:\n{randn}")
```

In [ ]:
```
# Create a tensor with a specific range
range_tensor = torch.arange(0, 10, step=1)
linspace = torch.linspace(0, 1, steps=5)
print(f"Range tensor: {range_tensor}")
print(f"Linspace tensor: {linspace}")

# Create an identity matrix
```

```
eye = torch.eye(3)
print(f"\nIdentity matrix:\n{eye}")
```

1.2 Tensor Attributes

In [ ]:
```
x = torch.randn(3, 4, 5)

print(f"Tensor shape: {x.shape}")
print(f"Tensor size: {x.size()}")
print(f"Number of dimensions: {x.dim()}")
print(f"Data type: {x.dtype}")
print(f"Device: {x.device}")
```

1.3 Tensor Indexing and Slicing

In [ ]:
```
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Original tensor:\n{x}")

# Indexing
print(f"\nIndexing:")
print(f"x[0, 0] = {x[0, 0]}")
print(f"x[1, 2] = {x[1, 2]}")

# Slicing
print(f"\nSlicing:")
print(f"First column:\n{x[:, 0]}")
print(f"Second row:\n{x[1, :]}")
print(f"Sub-matrix (top-right 2x2):\n{x[0:2, 1:3]}")
```

In [ ]:
```
# Advanced indexing
indices = torch.tensor([0, 2])
print(f"Advanced indexing with indices [0, 2]:\n{x[indices]}")

# Boolean indexing
mask = x > 5
print(f"\nBoolean mask (x > 5):\n{mask}")
print(f"Elements where x > 5:\n{x[mask]}")
```

2. Tensor Operations

PyTorch provides a wide range of operations for manipulating tensors.

2.1 Arithmetic Operations

In [ ]:
```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print(f"a = {a}")
print(f"b = {b}")

# Addition
print(f"a + b = {a + b}")
print(f"torch.add(a, b) = {torch.add(a, b)}")
```

```
# Subtraction
print(f"a - b = {a - b}")
print(f"torch.sub(a, b) = {torch.sub(a, b)}")

# Multiplication (element-wise)
print(f"a * b = {a * b}")
print(f"torch.mul(a, b) = {torch.mul(a, b)}")

# Division (element-wise)
print(f"a / b = {a / b}")
print(f"torch.div(a, b) = {torch.div(a, b)}")
```

In [ ]:
```
# In-place operations (modifies the tensor)
c = torch.tensor([1, 2, 3])
print(f"Original c = {c}")

c.add_(b)  # Note the underscore suffix for in-place operations
print(f"After c.add_(b), c = {c}")
```

2.2 Matrix Operations

In [ ]:
```
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])

print(f"Matrix a:\n{a}")
print(f"Matrix b:\n{b}")

# Matrix multiplication
print(f"\nMatrix multiplication (torch.matmul(a, b)):\n{torch.matmul(a, b
print(f"Matrix multiplication (a @ b):\n{a @ b}")

# Element-wise multiplication
print(f"\nElement-wise multiplication (a * b):\n{a * b}")

# Transpose
print(f"\nTranspose of a:\n{a.t()}")
```

In [ ]:
```
# Determinant
print(f"Determinant of a: {torch.det(a.float())}")

# Inverse
print(f"Inverse of a:\n{torch.inverse(a.float())}")
```

2.3 Reduction Operations

In [ ]:
```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(f"Tensor x:\n{x}")

# Sum
print(f"\nSum of all elements: {torch.sum(x)}")
print(f"Sum along rows (dim=0): {x.sum(dim=0)}")
print(f"Sum along columns (dim=1): {x.sum(dim=1)}")

# Mean
print(f"\nMean of all elements: {torch.mean(x.float())}")
print(f"Mean along rows (dim=0): {x.float().mean(dim=0)}")
print(f"Mean along columns (dim=1): {x.float().mean(dim=1)}")
```

In [ ]:
```
# Max and Min
print(f"Max of all elements: {torch.max(x)}")
max_values, max_indices = x.max(dim=0)
print(f"Max along rows (dim=0): values={max_values}, indices={max_indices
print(f"Min of all elements: {torch.min(x)}")

# Product
print(f"Product of all elements: {torch.prod(x)}")
```

2.4 Reshaping Operations

In [ ]:
```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(f"Original tensor x:\n{x}")

# Reshape
print(f"\nReshape to (3, 2):\n{x.reshape(3, 2)}")

# View (shares the same data with the original tensor)
print(f"\nView as (6, 1):\n{x.view(6, 1)}")

# Flatten
print(f"\nFlatten: {x.flatten()}")
```

In [ ]:
```
# Permute dimensions
y = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # Shape: (2, 2, 2
print(f"Original tensor y with shape {y.shape}:\n{y}")
print(f"\nPermute dimensions (2, 0, 1) with shape {y.permute(2, 0, 1).sha

# Squeeze and Unsqueeze
z = torch.tensor([[[1], [2]]])  # Shape: (1, 2, 1)
print(f"\nOriginal tensor z with shape {z.shape}:\n{z}")
print(f"Squeeze z with shape {z.squeeze().shape}: {z.squeeze()}")
print(f"Squeeze dimension 0 with shape {z.squeeze(0).shape}:\n{z.squeeze(
print(f"Unsqueeze x at dimension 0 with shape {torch.unsqueeze(x, 0).shap
```

3. NumPy Integration

PyTorch provides seamless integration with NumPy, allowing you to convert between PyTorch tensors and NumPy arrays.

In [ ]:
```
# Convert NumPy array to PyTorch tensor
np_array = np.array([1, 2, 3])
```

```
tensor = torch.from_numpy(np_array)
print(f"NumPy array: {np_array}")
print(f"PyTorch tensor from NumPy: {tensor}")

# Convert PyTorch tensor to NumPy array
tensor = torch.tensor([4, 5, 6])
np_array = tensor.numpy()
print(f"\nPyTorch tensor: {tensor}")
print(f"NumPy array from tensor: {np_array}")
```

In [ ]:
```
# Shared memory demonstration
np_array = np.array([1, 2, 3])
tensor = torch.from_numpy(np_array)
print(f"Original NumPy array: {np_array}")
print(f"Original tensor: {tensor}")

np_array[0] = 5
print(f"\nModified NumPy array: {np_array}")
print(f"Tensor after NumPy modification: {tensor}")
```

4. GPU Acceleration

One of the key features of PyTorch is its ability to leverage GPU acceleration for faster computations.

In [ ]:
```
# Check if CUDA (NVIDIA GPU) is available
cuda_available = torch.cuda.is_available()
print(f"CUDA available: {cuda_available}")

# Create tensors on CPU or GPU
if cuda_available:
    device = torch.device("cuda")
    print("Using CUDA device")

    # Create tensor directly on GPU
    x_gpu = torch.tensor([1, 2, 3], device=device)
    print(f"Tensor created on GPU: {x_gpu}")

    # Move tensor from CPU to GPU
    x_cpu = torch.tensor([4, 5, 6])
    x_gpu = x_cpu.to(device)
    print(f"Tensor moved from CPU to GPU: {x_gpu}")

    # Move tensor back to CPU
    x_cpu_again = x_gpu.cpu()
    print(f"Tensor moved back to CPU: {x_cpu_again}")
else:
    print("CUDA not available. Using CPU only.")
    device = torch.device("cpu")
    x = torch.tensor([1, 2, 3])
    print(f"Tensor on CPU: {x}")
```

5. Computational Graphs

PyTorch uses a dynamic computational graph, which means the graph is built on-the-fly as operations are executed.

In [ ]:
```
# Create tensors with requires_grad=True to track operations
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

print(f"x = {x}")
print(f"y = {y}")

# Build a computational graph
z = x**2 + y**3
print(f"z = x^2 + y^3 = {z}")

# Compute gradients
z.backward()

# Access gradients
print(f"Gradient of z with respect to x (dz/dx): {x.grad}")
print(f"Gradient of z with respect to y (dz/dy): {y.grad}")
```

In [ ]:
```
# Gradient accumulation
print("Gradient accumulation:")

# Reset gradients
x.grad.zero_()
y.grad.zero_()
print(f"After zeroing gradients:")
print(f"x.grad = {x.grad}")
print(f"y.grad = {y.grad}")

# Compute gradients multiple times
z = x**2 + y**3
z.backward()
print(f"\nAfter first backward pass:")
print(f"x.grad = {x.grad}")

z = x**2 + y**3
z.backward()
print(f"\nAfter second backward pass (gradients are accumulated):")
print(f"x.grad = {x.grad}")
```

In [ ]:
```
# Detach a tensor from the graph
a = x.detach()
print(f"Detached tensor a = {a}")
print(f"a.requires_grad = {a.requires_grad}")
```

6. Visualization Example

Let's visualize a simple function and its gradient using PyTorch.

In [ ]:
```
# Create a range of x values
x_range = torch.linspace(-3, 3, 100, requires_grad=True)

# Define a function: f(x) = x^2
y = x_range**2
```

```
# Compute gradients for each x value
gradients = []
for i in range(len(x_range)):
    if x_range.grad is not None:
        x_range.grad.zero_()
    y_i = x_range[i]**2
    y_i.backward(retain_graph=True)
    gradients.append(x_range.grad[i].item())

# Convert to NumPy for plotting
x_np = x_range.detach().numpy()
y_np = y.detach().numpy()
gradients_np = np.array(gradients)

# Plot the function and its gradient
plt.figure(figsize=(10, 6))
plt.plot(x_np, y_np, 'b-', label='f(x) = x^2')
plt.plot(x_np, gradients_np, 'r-', label="f'(x) = 2x")
plt.grid(True)
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.axvline(x=0, color='k', linestyle='-', alpha=0.3)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Function and its Gradient')
plt.legend()
plt.show()
```

Conclusion

This notebook covered the basics of PyTorch, including tensors, operations, NumPy integration, GPU acceleration, and computational graphs. These concepts form the foundation for building and training neural networks with PyTorch.

In the next tutorial, we'll explore automatic differentiation and optimization in more detail.